

Faculty of Computers and Information Systems

Information Retrieval

Practical Laboratory

Week 4: Term-Document Incidence Matrix

Represent the five-document collection using one binary row per term. This matrix becomes the input to Boolean retrieval in Week 5.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Assign consistent document IDs and a fixed column order.
- Construct a collection vocabulary from preprocessed text.
- Implement `build_matrix(documents)` using 0/1 occurrence values.
- Verify selected matrix rows manually.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. Complete the earlier labs before starting this week.

A term-document incidence matrix records whether a term occurs in each document. Rows correspond to terms; columns correspond to document IDs. A value of 1 means at least one occurrence, and 0 means absence. Repetition within one document does not change a binary entry.

The program stores the matrix as a dictionary from a term to a list of bits. The separate `doc_names` list specifies which document each position represents. Keep that list with the matrix: a vector without its column order is ambiguous.

Following 5.py, the program tokenizes each document, collects unique terms, and checks membership for every term/document pair. Sorted terms make printing repeatable; sorted document IDs make the vector interpretation stable. This straightforward representation is useful for learning, but a large sparse collection would store many zeros.

Course material: `build_matrix` and `print_matrix` in 5.py; the original five sample documents are retained.

3. Python Concepts Used

- **Dictionaries of lists:** Map each document to tokens, then each term to an occurrence row.
- **Nested loops:** Visit every term and every document.
- **set:** Remove duplicate terms when constructing the vocabulary.
- **Membership and conditional expressions:** Append 1 when a term occurs and 0 otherwise.
- **Multiple return values:** Return both matrix and `doc_names` so positions remain meaningful.

4. Worked Example

Build a tiny incidence matrix by hand before running the complete five-document program.

```
documents = {"d1": ["python", "data", "data"],
            "d2": ["java", "data"]}
for term in ["data", "java", "python"]:
    row = [1 if term in documents[name] else 0
           for name in sorted(documents)]
    print(term, row)
```

The data row is [1, 1], java is [0, 1], and python is [1, 0]. The repeated data token in d1 still contributes only one bit. In the supplied five-document collection, python occurs in doc1 and doc4.

Term	doc1	doc2	doc3	doc4	doc5
python	1	0	0	1	0
java	0	1	0	0	1
machine	1	0	1	0	0

5. Step-by-Step Practical Task

Step 1: Keep the supplied five documents unchanged for the first run.

Step 2: Tokenize each document using the Week 3 function.

Step 3: Build all_words, then convert it to a set and sort it to obtain the vocabulary.

Step 4: For each term, append one binary value for each ID in sorted(documents).

Step 5: Return the matrix and document order. Print labeled columns.

Step 6: Compare python, java, machine, and data rows with manual predictions, then complete the starter.

Execution: open a terminal in this week folder and run python lab04_solution.py. On Windows, py -3 can replace python. All required files are included; no internet or third-party Python packages are needed.

6. Complete Python Program

The listing below matches lab04_solution.py exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

lab04_solution.py

```
import os
import re

STOP_WORDS = {
    "a", "an", "the", "is", "in", "for", "and", "are", "with",
    "like", "of", "to", "it", "be", "used", "uses", "that", "this",
    "was", "were", "by", "from", "on", "at", "or",
}

SAMPLE_DOCUMENTS = {
    "doc1": "Python is great for data science and machine learning",
    "doc2": "Java is used for enterprise applications and web development",
    "doc3": "Machine learning uses neural networks and decision trees",
    "doc4": "Python supports data analysis with libraries like Pandas",
    "doc5": "Java and C++ are used for game development and graphics",
}

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def load_documents(folder_path):
    """Load UTF-8 .txt files; use five samples if missing or empty."""
    documents = {}
    if os.path.isdir(folder_path):
        for filename in sorted(os.listdir(folder_path)):
            filepath = os.path.join(folder_path, filename)
            if filename.endswith(".txt") and os.path.isfile(filepath):
                with open(filepath, "r", encoding="utf-8") as file:
                    documents[filename[:-4]] = file.read()
    if not documents:
```

```

        return SAMPLE_DOCUMENTS.copy()
    return documents

def tokenize(text):
    """Use the same ASCII-letter tokens as the supplied course files."""
    words = re.findall(r"[a-z]+", text.lower())
    return [word for word in words if word not in STOP_WORDS]

def build_matrix(documents):
    doc_names = sorted(documents)
    words_in_docs = {}
    all_words = []
    for name in doc_names:
        words_in_docs[name] = tokenize(documents[name])
        all_words.extend(words_in_docs[name])
    matrix = {}
    for term in sorted(set(all_words)):
        row = []
        for name in doc_names:
            row.append(1 if term in words_in_docs[name] else 0)
        matrix[term] = row
    return matrix, doc_names

def print_matrix(matrix, doc_names):
    print("Term".ljust(14), "|", " | ".join(doc_names))
    for term, row in matrix.items():
        values = " | ".join(str(bit).center(4) for bit in row)
        print(term.ljust(14), "|", values)

def main():
    documents = load_documents(os.path.join(BASE_DIR, "docs"))
    matrix, doc_names = build_matrix(documents)
    print_matrix(matrix, doc_names)

if __name__ == "__main__":
    main()

```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; `expected_output.txt` contains the full terminal output. Selected rows are shown for large matrices.

Term	doc1	doc2	doc3	doc4	doc5
analysis	0	0	0	1	0
data	1	0	0	1	0
java	0	1	0	0	1
machine	1	0	1	0	0
python	1	0	0	1	0

8. Laboratory Exercises

1. Basic: predict the rows for learning and analysis before running the program.
2. Implementation: complete `build_matrix` in the starter, including vocabulary construction.
3. Debugging: deliberately reverse the printed column labels without reversing the vectors. Explain the incorrect interpretation this produces.
4. Interpretation: add three more occurrences of `python` to `doc1`. State which matrix entries change and why.
5. Implementation: write a function that returns the document IDs for a given term row.
6. Extension: add a sixth document and verify that every row has exactly six values.

9. Student Tasks

- Your completed matrix builder and a labeled matrix for the original five documents.
- Manual predictions and checks for four selected terms.
- A short comparison of a token count and a binary occurrence value.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. What does one row represent?
2. Why must document order remain fixed?
3. What value represents repeated occurrences of a term in one document?
4. Why does vocabulary construction use a set?
5. Why can an incidence matrix waste space for a large collection?

11. Challenge Exercise

Optional: Count the number of 1s and 0s and calculate the proportion of nonzero entries. Explain why this density measure matters when choosing a representation.